CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

Data Engineering (ITUC301)

Practical File – ODD Semester (2026-27)

Student ID:24DIT043

Student Name : Krushil Patel

PRACTICAL-3

Problem Definition:

- Design, spin up, and evaluate two parallel data ingestion pipelines: a periodic time-triggered block batch pipeline and a continuous low-latency real-time event streaming processor. Students must capture telemetry statistics and benchmark performance parameters across both options, measuring data delivery latency, processing throughput spikes, and fault tolerance during simulated infrastructure drops.

Implementation :

- **Files Created :**

User_activity.csv:

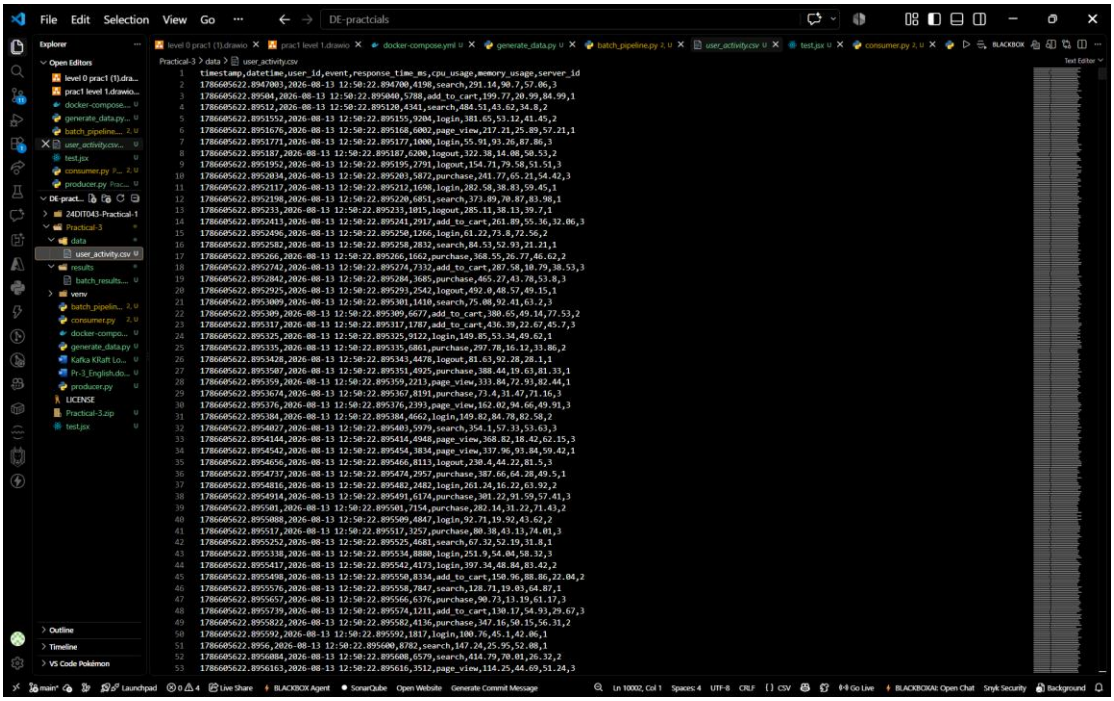


Figure 1.1) User activity

Batch Processing :

Batch_pipeline.py :

```
import pandas as pd
import time
import os
```

```
# Configuration
DATA_FILE = "data/user_activity.csv"

# Number of records processed in one batch
BATCH_SIZE = 1000

print("========================================")
print("       BATCH DATA PROCESSING")
print("========================================")

# Check whether dataset exists
if not os.path.exists(DATA_FILE):
    print("Dataset not found!")
    print("Run generate_data.py first.")
    exit()

# Read dataset
df = pd.read_csv(DATA_FILE)

print("Total records:", len(df))
print("Batch size:", BATCH_SIZE)
print()

# Store benchmark results
results = []

total_records = len(df)

batch_number = 0

overall_start = time.time()

# Process data in blocks
for start in range(0, total_records, BATCH_SIZE):

    batch_number += 1
```

```
    end = min(start + BATCH_SIZE, total_records)

    batch = df.iloc[start:end].copy()

    print("----------------------------------------")
    print("Processing Batch:", batch_number)
    print("Records:", len(batch))

    # Start processing timer
    processing_start = time.time()

    # Simulate batch processing

    # Average response time
    average_response_time = batch["response_time_ms"].mean()

    # Average CPU usage
    average_cpu = batch["cpu_usage"].mean()

    # Average memory usage
    average_memory = batch["memory_usage"].mean()

    # Count events
    event_counts = batch["event"].value_counts()

    # End processing timer
    processing_end = time.time()

    processing_time = processing_end - processing_start

    # Calculate throughput
    throughput = len(batch) / processing_time if processing_time > 0
else 0

    print(
        "Processing Time:",
        round(processing_time, 6),
        "seconds"
```

```
    )

    print(
        "Throughput:",
        round(throughput, 2),
        "records/sec"
    )

    print(
        "Average Response Time:",
        round(average_response_time, 2),
        "ms"
    )

    print(
        "Average CPU:",
        round(average_cpu, 2),
        "%"
    )

    print(
        "Average Memory:",
        round(average_memory, 2),
        "%"
    )

    # Store result
    results.append({
        "batch": batch_number,
        "records": len(batch),
        "processing_time_sec": processing_time,
        "throughput_records_sec": throughput,
        "average_response_time_ms": average_response_time,
        "average_cpu": average_cpu,
        "average_memory": average_memory
    })

overall_end = time.time()
```

```
total_time = overall_end - overall_start

# Convert results to DataFrame
results_df = pd.DataFrame(results)

# Overall statistics
total_throughput = total_records / total_time

print("\n========================================")
print("          BATCH RESULTS")
print("========================================")

print("Total Records:", total_records)

print(
    "Total Processing Time:",
    round(total_time, 4),
    "seconds"
)

print(
    "Overall Throughput:",
    round(total_throughput, 2),
    "records/sec"
)

print(
    "Average Batch Throughput:",
    round(
        results_df["throughput_records_sec"].mean(),
        2
    ),
    "records/sec"
)

# Save benchmark results
os.makedirs("results", exist_ok=True)
```

```
results_file = "results/batch_results.csv"

results_df.to_csv(results_file, index=False)

print("\nBenchmark results saved to:")
print(results_file)

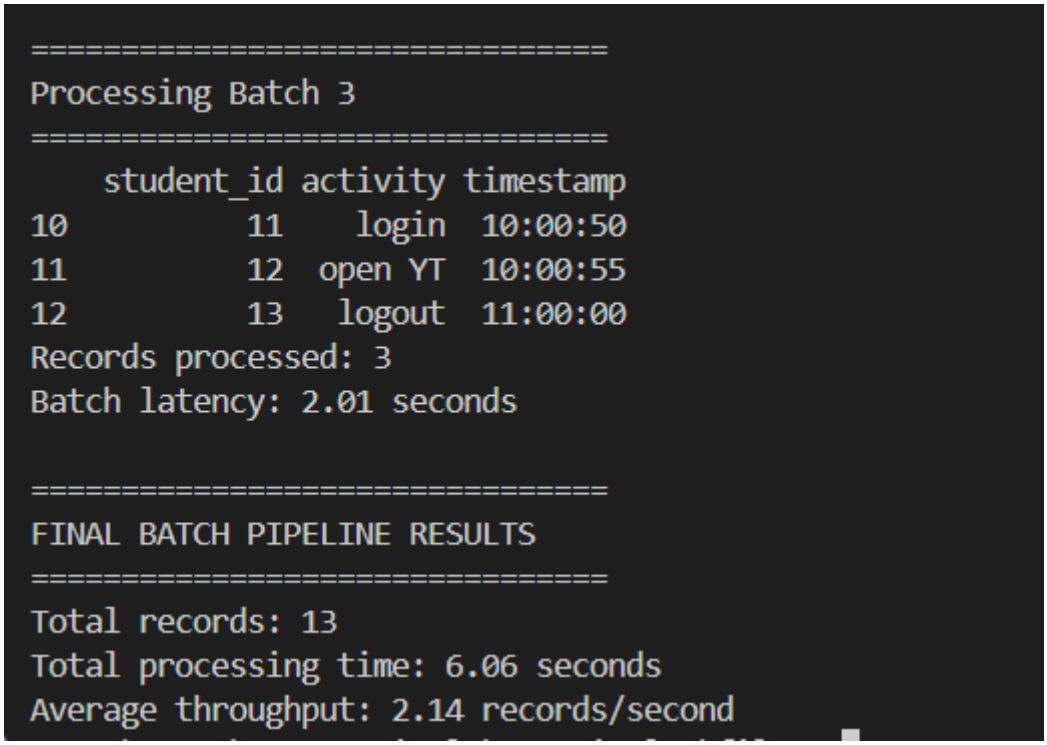
print("\nBatch processing completed!")
```

OUTPUT :

```
PS F:\sem 5\DE_practicals\practical 3\files> python batch_pipeline.py
Starting Batch Pipeline...
Total records: 13

==============================
Processing Batch 1
==============================
   student_id            activity timestamp
0           1               login  10:00:01
1           2             open YT  10:00:10
2           3  played online game  10:00:11
3           4               login  10:00:20
4           5     change password  10:00:25
Records processed: 5
Batch latency: 2.02 seconds

==============================
Processing Batch 2
==============================
   student_id              activity timestamp
5           6  assignment_submission  10:00:40
6           7                 login  10:00:42
7           8               logiout  10:00:44
8           9             oprn Insta  10:00:48
9          10           open chatgpt  10:00:50
Records processed: 5
Batch latency: 2.03 seconds
```



consumers.py :

```
from kafka import KafkaConsumer
import json
import time

# ==============================
# CONFIGURATION
# ==============================

KAFKA_SERVER = "localhost:9092"
TOPIC = "web_activity"
GROUP_ID = "web-activity-group"

TOTAL_MESSAGES = 10000

# ==============================
# CREATE CONSUMER
# ==============================
```

```
consumer = KafkaConsumer(
    TOPIC,
    bootstrap_servers=KAFKA_SERVER,

    # Start from earliest message
    auto_offset_reset="earliest",

    # Consumer group
    group_id=GROUP_ID,

    # Convert JSON back to Python dictionary
    value_deserializer=lambda x: json.loads(
        x.decode("utf-8")
    )
)

print("========================================")
print("      KAFKA STREAMING CONSUMER")
print("========================================")

print("Topic:", TOPIC)
print("Waiting for messages...\n")

# ================================
# VARIABLES
# ================================

count = 0

total_latency = 0

max_latency = 0

start_time = time.time()
```

```
# ================================
# RECEIVE MESSAGES
# ================================

for message in consumer:

    data = message.value

    receive_time = time.time()

    # Calculate end-to-end Kafka delivery latency
    latency = (
        receive_time -
        data["producer_timestamp"]
    ) * 1000

    total_latency += latency

    max_latency = max(max_latency, latency)

    count += 1

    # Progress
    if count % 1000 == 0:

        print(
            "Messages received:",
            count,
            "| Latest latency:",
            round(latency, 2),
            "ms"
        )
```

```
    # Stop after all records
    if count >= TOTAL_MESSAGES:

        end_time = time.time()

        break

# ================================
# CALCULATE RESULTS
# ================================

total_time = end_time - start_time

average_latency = total_latency / count

throughput = count / total_time if total_time > 0 else 0

# ================================
# DISPLAY RESULTS
# ================================

print("\n========================================")
print("      STREAMING RESULTS")
print("========================================")

print("Total Messages:", count)

print(
    "Total Time:",
    round(total_time, 4),
    "seconds"
)

print(
```

```
    "Average Latency:",
    round(average_latency, 2),
    "ms"
)

print(
    "Maximum Latency:",
    round(max_latency, 2),
    "ms"
)

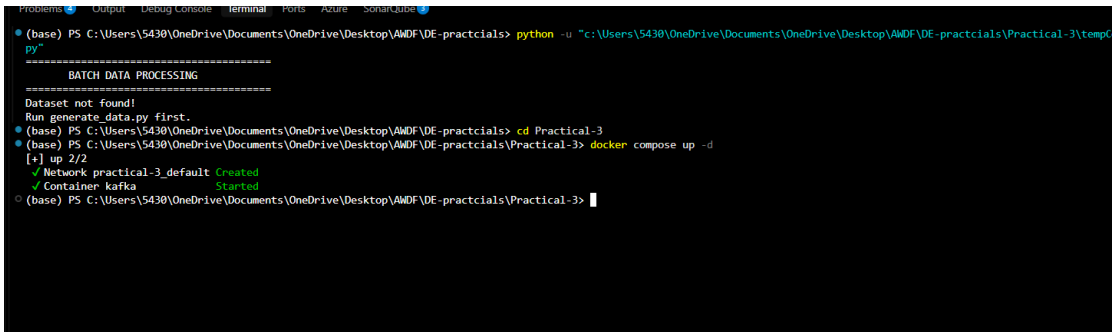
print(
    "Throughput:",
    round(throughput, 2),
    "messages/sec"
)

consumer.close()

print("\nConsumer finished.")
```

- **OUTPUT :**

Starting the kafka using the docker



Running both the files :

```
PS F:\sem 5\DE_practicals\practical 3\files> python producers.py
Dynamic Kafka Producer Started...
Press CTRL + C to stop.

Sent: {'event_id': 1, 'student_id': 103, 'activity': 'video_watched', 'timestamp': '2026-08-25T20:29:10.518647'}
Sent: {'event_id': 2, 'student_id': 101, 'activity': 'logout', 'timestamp': '2026-08-25T20:29:12.571465'}
Sent: {'event_id': 3, 'student_id': 103, 'activity': 'logout', 'timestamp': '2026-08-25T20:29:14.025047'}
Sent: {'event_id': 4, 'student_id': 105, 'activity': 'login', 'timestamp': '2026-08-25T20:29:15.554571'}
Sent: {'event_id': 5, 'student_id': 101, 'activity': 'quiz_attempt', 'timestamp': '2026-08-25T20:29:16.785665'}
Sent: {'event_id': 6, 'student_id': 103, 'activity': 'video_watched', 'timestamp': '2026-08-25T20:29:17.549753'}
Sent: {'event_id': 7, 'student_id': 101, 'activity': 'video_watched', 'timestamp': '2026-08-25T20:29:18.594457'}
Sent: {'event_id': 8, 'student_id': 104, 'activity': 'login', 'timestamp': '2026-08-25T20:29:20.405831'}
Sent: {'event_id': 9, 'student_id': 103, 'activity': 'video_watched', 'timestamp': '2026-08-25T20:29:21.135051'}
Sent: {'event_id': 10, 'student_id': 101, 'activity': 'logout', 'timestamp': '2026-08-25T20:29:21.772610'}
Sent: {'event_id': 11, 'student_id': 101, 'activity': 'assignment_submission', 'timestamp': '2026-08-25T20:29:23.373450'}
Sent: {'event_id': 12, 'student_id': 104, 'activity': 'assignment_submission', 'timestamp': '2026-08-25T20:29:25.045835'}
Sent: {'event_id': 13, 'student_id': 101, 'activity': 'assignment_submission', 'timestamp': '2026-08-25T20:29:26.382158'}
Sent: {'event_id': 14, 'student_id': 105, 'activity': 'video_watched', 'timestamp': '2026-08-25T20:29:27.832475'}

Stopping Producer...
```

```
PS F:\sem 5\DE_practicals\practical 3\files> python consumers.py
 Dynamic Kafka Consumer Started...
 Waiting for live events...

 ----------------------------------
 Event ID: 1
 Student ID: 103
 Activity: video_watched
 Sent Time: 2026-08-25 20:29:10.518647
 Received Time: 2026-08-25 20:29:10.580491
 Latency: 61.84 ms
 Total Messages: 1
 Throughput: 0.08 messages/sec
 ----------------------------------
 ----------------------------------
 Event ID: 2
 Student ID: 101
 Activity: logout
 Sent Time: 2026-08-25 20:29:12.571465
 Received Time: 2026-08-25 20:29:12.578004
 Latency: 6.54 ms
 Total Messages: 2
 Throughput: 0.15 messages/sec
 ----------------------------------
 ----------------------------------
 Event ID: 3
 Student ID: 103
 Activity: logout
 Sent Time: 2026-08-25 20:29:14.025047
 Received Time: 2026-08-25 20:29:14.035147
 Latency: 10.1 ms
 Total Messages: 3
```

Key Questions:

1.) In what scenarios does a micro-batch architecture provide a better operational balance of performance and complexity than a true real-time streaming approach**?**

Ans.) Micro-batching is preferred when a **small delay in processing is acceptable** and there is no need for immediate, event-by-event processing. It collects data for a short period and processes it as a small batch. This reduces the complexity and overhead of maintaining a true real-time streaming system while still providing relatively fast results. It is suitable for **log analysis, periodic monitoring, reporting, ETL pipelines, and analytics**, where delays of a few seconds or minutes are acceptable.

2.) How does an event streaming broker maintain message delivery guarantees during abrupt broker cluster node disconnects?

Ans.) An event streaming broker such as **Apache Kafka** maintains message delivery through **data replication, acknowledgments, leader election, and consumer offsets**. Each topic partition can have multiple replicas stored on different brokers. If the broker acting as the leader fails, another replica can be elected as the new leader. Producers can use acknowledgments to ensure messages have been successfully replicated, while consumers use offsets to track which messages have already been processed. This allows the system to **continue operating and minimize data loss or duplicate processing** even when a broker suddenly disconnects.

3.) Detail the network and compute resource trade-offs between bulk file compression operations and individual event serialization overheads.

Ans.) Bulk file compression processes large amounts of data together, which can significantly **reduce network bandwidth usage** because compressed files are smaller. However, compression requires considerable **CPU resources** and may introduce processing delays. In contrast, **individual event serialization** converts each event into a transferable format immediately, providing **lower latency and faster event delivery**, which is useful for real-time systems. However, processing and transmitting every event separately creates **continuous CPU, memory, and network overhead**. Therefore, bulk compression is generally better for **high-volume batch transfers**,

while individual serialization is more suitable for **low-latency streaming applications**

Supplementary Problems:

- Extend your streaming pipeline to handle sudden traffic spikes by scaling your message broker topic partition layers and verifying how consumer groups scale out to process the load.

Solution :

To handle sudden traffic spikes in a streaming pipeline, the Kafka topic can be scaled by increasing its number of partitions. Partitions allow incoming messages to be distributed across multiple processing units, so more partitions can increase the overall throughput.

The consumer group can then scale out by adding more consumers. Kafka assigns different partitions to different consumers within the same group, allowing multiple messages to be processed in parallel. For example, if a topic has 6 partitions and the consumer group has 3 consumers, each consumer can process approximately 2 partitions.

Therefore, increasing topic partitions + consumer instances helps the system handle higher traffic efficiently. However, the number of active consumers should not exceed the number of partitions, because extra consumers will remain idle. This provides parallel processing, higher throughput, and better handling of sudden traffic spikes.

Conclusion :

This practical demonstrated how the choice of a data ingestion method affects the speed, scalability, and processing requirements of a data pipeline. Batch processing provides a simpler approach for handling accumulated data at scheduled intervals, whereas Kafka enables continuous event processing with better responsiveness. The experiment also showed that Kafka can distribute workloads across partitions and consumers, making it suitable for systems where the data rate may increase

unexpectedly. Hence, the appropriate ingestion approach depends on the application's latency requirements, data volume, and scalability needs..